

Safety Analysis of Autonomous Driving Systems: A Simulation-based Runtime Verification Approach

Duong Dinh Tran, Takashi Tomita, and Toshiaki Aoki

Japan Advanced Institute of Science and Technology, Ishikawa 923-1292, Japan

Abstract—Ensuring the safety of Autonomous Driving Systems (ADSs) through rigorous verification in simulated environments is crucial before real-world deployment. However, using simulation environments for ADS testing and verification poses several challenges, including specifying various behaviors of traffic participants and collecting comprehensive real-time data to verify the ADS. To address these challenges, we propose a framework for runtime verification of ADSs, focusing on Autoware, a leading ADS. This framework integrates AWSIM-Script, a scripting language for defining traffic scenarios; Runtime Monitor, a tool to record real-time data during the simulation; and AW-Checker, a Linear Temporal Logic-based property checker for verifying safety requirements. Unlike prior research that primarily focuses on generating critical scenarios, we leverage a well-established ADS safety standard from the Japan Automobile Manufacturers Association and adopt its systematic methodology for safety assessment. We conducted a series of experiments focused on non-intersection road geometry to evaluate Autoware’s capability in handling different traffic disturbances such as cut-in, cut-out, and deceleration scenarios. The results revealed that, compared to the competent and careful driver model, which represents the minimum safety requirements for ADSs, Autoware failed to prevent collisions in some cases, particularly during high-speed scenarios and fast lateral movements by other vehicles.

Index Terms—autonomous driving systems, safety verification, Autoware, safety standard, model checking

I. INTRODUCTION

AUTONOMOUS Driving Systems (ADSs) are emerging as a promising means of transportation for the future. Due to the inherent complexity of these systems, rigorous testing and verification are essential to ensure their safe and reliable operation under a wide range of real-world conditions. Testing ADSs can be performed in real-world or simulated environments. Although real-world testing provides highly realistic scenarios, it is resource-intensive and often dangerous, particularly for risk cases, such as a vehicle suddenly stops. As an alternative, simulation environments play an indispensable role in ADS development.

Simulators like CARLA [1], SVL [2], and AirSim [3] provide controlled and customizable environments where different traffic scenarios can be realized without the risks and costs as in the real-world environment. However, several challenges arise when using simulators to evaluate the operation of ADSs. The first challenge lies in the simulator’s ability to control

and simulate participants’ behaviors, allowing comprehensive variations of driving situations. The second challenge is how real-time data during simulation and ADS execution can be collected to facilitate thorough verification. The third challenge is how to verify that the ADS behaves as expected and meets the desired safety requirements based on the collected data.

There exist a number of studies on scenario-based testing and verification of ADSs, such as [4]–[11], but they primarily focus on identifying and/or generating critical cases that pose significant risks to ADSs. While these approaches are valuable for detecting high-risk scenarios, they cannot guarantee comprehensive coverage of all potential cases. To ensure ADS safety before public road deployment, it is necessary to account for all possible variations the ego vehicle might encounter. To achieve this, some well-established ADS safety standards, such as [12], [13], provide a structured and systematic methodology for thorough safety analysis. How these standards can be utilized to perform safety assessments on real-world ADSs is the central research question that we aim to answer in this work.

Unlike prior studies that concentrate on critical scenario generation, we leverage the Automated Driving Safety Evaluation Standard proposed by the Japan Automobile Manufacturers Association (JAMA) [12] (hereinafter referred to as JAMA standard) to perform ADS safety analysis. To support this analysis, we introduce a simulation-based runtime verification framework, comprising AWSIM-Script for traffic scenario description, Runtime Monitor for capturing detailed simulation data, and AW-Checker for property checking based on Linear Temporal Logic (LTL). In this work, we implement the framework with Autoware [14], a leading open-source platform for autonomous driving, and the AWSIM-Labs simulator [15]. Autoware and Baidu Apollo [16] are the only open-source autonomous driving platforms supporting full autonomy. While there exist many studies subject to Baidu Apollo safety analysis, such as [5], [7], [9]–[11], [17], [18], to the best of our knowledge, this study is the first to attempt either verification or automated testing of Autoware.

AWSIM-Script is a scripting language designed to simplify the specification of simulation scenarios. Scenarios required for safety assessments are extracted from the safety standard and specified as individual script files. Each script is then processed by AWSIM-Script to execute the corresponding scenario. The language enables users to define the behaviors of traffic participants within a single input script, enhancing the efficiency of simulations and enabling seamless verification in subsequent steps.

Manuscript received October 23, 2024; revised January 27, 2025; accepted April 14, 2025. This work was supported by JST, CREST Grant Number JPMJCR23M1.

The authors are with Japan Advanced Institute of Science and Technology, Japan (emails: {duongtd, tomita, toshiaki}@jaist.ac.jp).

The Runtime Monitor records real-time data on the ego vehicle, traffic participants, and internal status of the ADS during execution, and dumps this data into a trace file for offline verification. Safety requirements, which are formalized as a set of properties in LTL, are then verified against this trace using AW-Checker, which leverages the Maude LTL model checker [19]. The monitor also collects data produced by each ADS sub-module, such as detected vehicles from the perception module and planning trajectories from the planning module. As a result, this framework enables both system-level verification, such as collision avoidance, and component-level verification, such as how precise the perception system detects surrounding vehicles. If a property is violated, the analysis results can let us know which module is responsible for the issue.

The JAMA standard [12] takes into account comprehensive variations of all factors that may affect the ego vehicle's operation, such as, road geometry variations and position as well as behavior variations of surrounding vehicles. All of these variations are combined and each combination instance serves as a scenario for evaluating ADS performance. In contrast to prior work that primarily focuses on identifying and generating critical scenarios using self-defined safety metrics, we utilize the JAMA methodology, which provides comprehensive scenarios required to assess how ADS behave, the challenging scenarios among them, and the safety benchmarks. To the best of our knowledge, we are the first to use the JAMA standard to conduct safety testing or verification of ADSs.

AWSIM-Script and AW-Checker facilitate the simulation of scenarios and the verification of safety requirements defined by the JAMA standard. We conducted a series of experiments to assess the safety of Autoware under different traffic disturbance variations defined by the JAMA standard in non-intersection road geometries, such as vehicles cutting in, cutting out, and decelerating. While no collision would occur under the *competent and careful human driver model*, which simulates a highly skilled and cautious human driver, Autoware failed to prevent collisions in some cases. JAMA established this *competent and careful driver model* as the minimum safety benchmark. An ADS must show a higher level of hazard detection and collision avoidance performance than this model; otherwise, the ADS is unsafe.

The key contributions of this study are as follows:

- We introduced the scenario description language AWSIM-Script, allowing for greater simulation controllability and simplifying the process of describing traffic scenarios. Specifically, AWSIM-Script supports the specification of scenarios defined in the JAMA standard.
- We developed a runtime monitor to record comprehensive simulation data, facilitating offline verification through trace analysis.
- We implemented AW-Checker to verify various safety properties including those defined in the JAMA standard. AW-Checker allows for the verification of both system-level properties (e.g., non-collision) and component-level properties (e.g., precision of the perception).
- Employing the JAMA standard's competent and careful driver model as a safety benchmark, we conducted a

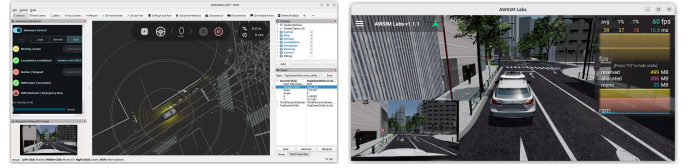


Fig. 1. Autoware and AWSIM-Labs GUIs.

series of experiments to evaluate Autoware's performance under various traffic disturbances on non-intersection roads. The experimental results revealed several weaknesses of Autoware, most notably its underperformance in collision avoidance compared to the careful driver model.

The framework's source code, materials supporting the experiments with Autoware, and setup guidelines are available at <https://github.com/duongtd23/AW-Runtime-Verification>.

The remainder of this paper is organized as follows. Section II introduces the background and provides an overview of our verification framework. In Section III, we elaborate on the extended AWSIM-Labs and AWSIM-Script. In Section IV, we present the Runtime Monitor and AW-Checker, demonstrating how we can use them for verifying desired properties. Section V reports the experimental results that we conducted according to the JAMA standard. Section VI discusses some closely related work, and Section VII summarizes our study.

II. BACKGROUND AND FRAMEWORK OVERVIEW

In this section, we give a brief introduction to Autoware, AWSIM-Labs, and the Automated Driving Safety Evaluation Standard by JAMA. We then present an overview of the proposed verification framework.

A. Autoware and AWSIM-Labs

Autoware is one of the first “all-in-one” and open-source solutions for developing and operating autonomous driving systems all over the world. The development of this open-source platform was originally led by TIER IV, an autonomous technology startup. TIER IV continues to advance Autoware's adoption by offering a robust set of tools and resources for developing, deploying, and maintaining autonomous vehicles, including the so-called AWSIM simulator, which allows developers to test and validate their systems in virtual environments.

AWSIM-Labs, a fork of AWSIM maintained by the Autoware Foundation, offers additional features while requiring fewer hardware resources, such as GPU computation. Both AWSIM and AWSIM-Labs are built on the Unity Engine. Key enhancements in AWSIM-Labs include the ability to reset vehicle positions during runtime, support for multiple scenes as well as vehicle setups, and various sensor models. Figure 1 shows the graphical user interfaces (GUI) of AWSIM-Labs and Autoware.

Communication between Autoware and AWSIM-Labs is facilitated through Robot Operating System 2 (ROS 2) [20]. Through ROS 2, AWSIM-Labs sends information, such as, the simulated sensor data to Autoware. Meanwhile, Autoware sends control commands to AWSIM-Labs, from which the simulator updates the ego's location, velocity, and many others.

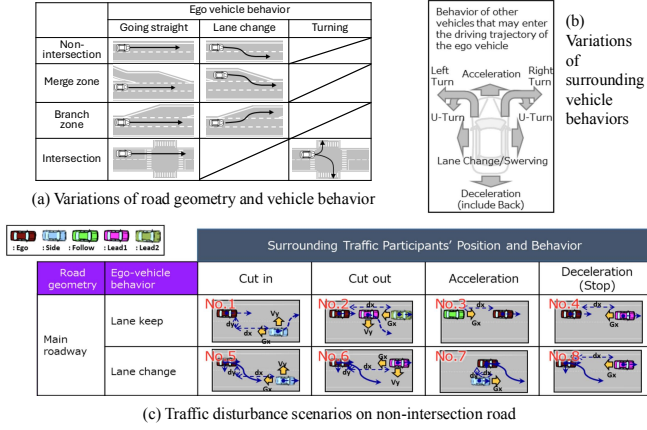


Fig. 2. Traffic disturbance variations and scenario examples [12].

B. ADS Safety Standard by Japan Automobile Manufacturers Association

The JAMA standard provides a structured methodology for evaluating the safety of ADSs in various driving scenarios. The safety is evaluated considering three types of disturbances: *perception*, *traffic*, and *vehicle*. Perception disturbance involves sensor misjudgments due to factors like environmental conditions or sensor placement. Traffic disturbance arises from interactions with other vehicles or complex road layouts, such as sudden lane changes of other vehicles. Vehicle disturbance occurs when the ADS struggles to control the vehicle's dynamics due to factors like weight distribution or road surface conditions. In this study, we focus on the traffic disturbance.

Comprehensive variations of traffic factors that may affect the ego vehicle are taken into consideration. Such variations include, for example, road geometry, which is either intersection, non-intersection, merge zone, or branch zone (Figure 2 (a)), surrounding Non-Player Character (NPC) relative positions like front, left, and right, and surrounding NPC behaviors, such as, accelerating, decelerating, and turning (Figure 2 (b)). All these variations are then combined together to create a detailed matrix, with each matrix element being an instance of the combination and serving as a scenario to evaluate the ADS. For instance, Figure 2 (c) depicts eight scenarios that need to be considered on non-intersection roads and U-turns are not allowed. In scenario No. 1 (top left of the figure), the NPC vehicle (blue color) makes a cut-in with lateral velocity V_y when the longitudinal and lateral distances to the ego vehicle (red color) are dx and dy , respectively.

Competent and careful human driver model. JAMA introduces the careful driver model as a safety benchmark for assessing ADSs. This driver model simulates how a careful human driver with perfectly excellent driving skills detects hazards, makes decisions, and takes actions such as braking to avoid accidents. The ADS under test/verification must show a higher level of hazard avoidance performance than this driver model within a foreseeable range of situations.

The careful driver model is built around the core elements of *risk perception*, *risk judgment*, and *operation delay*. The risk perception step refers to the time it takes for the driver

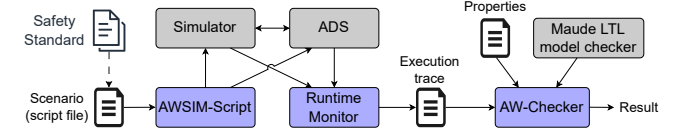


Fig. 3. Framework overview.

to identify a potential hazard in their environment. The risk judgment step refers to the time needed for the driver to assess the risk and determine the necessity of an emergency brake. The operation delay accounts for the time between deciding to brake and applying it, including the decision-making delay, releasing the accelerator pedal, transferring the foot to the brake pedal, and activating the brake. In the cut-in scenario, risk perception is defined as the duration for the cut-in vehicle to complete a lateral movement of 0.375 meters, while risk judgment and operation delay are 0.4 seconds and 0.75 seconds, respectively. These values are not defined by themselves but are derived from reliable existing studies or analyses of real statistical data.

C. Framework Overview

Figure 3 depicts the overall architecture of the proposed verification framework. The blue blocks represent newly developed components, while the gray blocks indicate existing systems employed in our framework.

AWSIM-Script is a scripting language for describing traffic scenarios to be executed (Section III-B). Scenarios required for safety assessments are derived from the safety standard and specified as individual script files. These scripts are passed to AWSIM-Script, which executes the corresponding scenarios. AWSIM-Script offers users the flexibility to configure various aspects of the simulation, including traffic participants and their planned trajectories. Additionally, the script allows users to define key parameters for the ego vehicle, such as its initial pose and goal, simplifying scenario creation in a single script.

Once a simulation is executed, the Runtime Monitor records the real-time data from the simulation environment and outputs this information into a trace file once the simulation finishes (Section IV-A). The monitor captures comprehensive data, including both the ground truth trajectories of the ego and NPCs in the simulated environment and the internal decisions of the ADS (e.g., planning trajectory for the ego vehicle), enabling component-level verification for each ADS module. The trace file allows for offline analysis and verification of desired properties without rerunning the simulation, saving time and resources. The trace data can be output in multiple formats, either as a Maude state machine for directly performing safety verification with AW-Checker or in a widely used format like YAML for other analysis approaches.

The AW-Checker (Section IV-B) takes an execution trace and the properties of interest as the inputs. The properties are in the form of LTL formulas, which are constructed from a set of predefined atomic propositions, representing conditions to be checked against the trace. We use Maude as the specification and programming language to define the syntax and semantics of these atomic propositions, respectively. Using

the Maude LTL model checker, AW-Checker verifies whether the trace satisfies the given properties. Leveraging the Maude LTL model checker offers two key benefits: it provides high performance and helps avoid potential bugs that could arise from self-implemented search/checking algorithms.

III. AWSIM-SCRIPT: A SCENARIO DESCRIPTION LANGUAGE

The implementation of AWSIM-Script comprises two components: a parser, which parses a user-provided input script into an internal structure, and a simulator adapter, which initializes the scenario in the simulator. While the parser is simulator-independent and can be reused across different simulation environments, the adapter is tailored to the specific simulator used. In this work, we implemented AWSIM-Script using the AWSIM-Labs simulator. However, the original AWSIM-Labs lacked support for certain fundamental traffic participant behaviors, which made it impossible to realize the scenarios required for evaluation. Hence, we first extended it to mitigate these limitations.

A. Extended AWSIM-Labs with custom behaviors

To simulate the traffic environment, AWSIM-Labs in its original form supports only two kinds of NPC vehicles:

- *Random NPCs*: They are randomly generated at arbitrary locations within the simulated environment. Once spawned, a random NPC takes a random route to move around and may magically disappear at some moment.
- *Defined route-based NPCs*: They follow a predefined route, that is, a sequence of traffic lanes. The NPC will be spawned at the starting point of the first traffic lane. They disappear once they reach the end of the last lane.

These two types of NPCs have notable limitations. Users lack the ability to control critical aspects, such as stopping an NPC at a specific location, making a lane change at a designated point, or setting an NPC as a stationary obstacle. Additionally, all NPC vehicles always follow speed limits assigned to traffic lanes and have fixed acceleration and deceleration values. This lack of flexibility prevents the simulation of different traffic scenarios, such as sudden stops or lane changes, which are essential for comprehensive ADS testing.

To mitigate these limitations, we extended AWSIM-Labs to support more advanced features. Some key features can be summarized as follows:

- Custom spawning position and goal: NPC vehicles and pedestrians can be configured to spawn at specific points (with orientation) and stop once they reach the defined goal positions.
- Stationary NPCs: NPC vehicles and pedestrians can be configured to be static obstacles during the simulation.
- Controllable trajectories: Users can define NPC vehicle trajectories by specifying a list of lanes to follow, speed limits on each lane, and lane change details, including the location to start lane change and the lateral velocity. A pedestrian traveling path can be set using a sequence of waypoints.

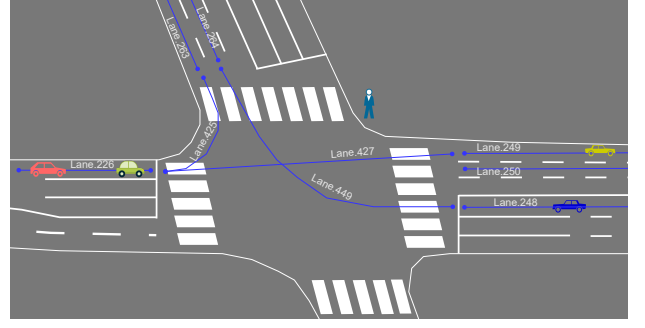


Fig. 4. The initial scene corresponding to the script in Listing 1.

- *Dynamic acceleration and deceleration*: Different acceleration and deceleration profiles can be assigned to each NPC vehicle.
- *Motion delay controls*: Customizable delays for spawning and starting NPC motion can be set, based on specific conditions such as the ego vehicle's movement.

These extended features enhance simulation flexibility, enabling the creation of a broader range of traffic scenarios.

B. AWSIM-Script

AWSIM-Script provides an interface for specifying NPC behaviors, ego vehicle configurations, and simulation scenarios. Listing 1 presents a script example. The simulation made from this input script has the initial scene shown in Figure 4. This running example introduces the participation of three NPC vehicles `npc1`, `npc2`, and `npc3` (green, blue, and yellow cars, respectively) and a pedestrian `pedes` along with the ego vehicle (red). The blue lines in the figure denote traffic lanes. The script example specifies the behaviors of these NPCs and setups for the ego as follows:

- `npc1` is spawned on lane 226, 20 meters from the start point of the lane. It follows two lanes 226 and 427 at speeds of 10 m/s and 8.33 m/s, respectively, passing the intersection and reaching lane 249. After traveling 8 meters on lane 249, it starts making a lane change to the adjacent lane 250 with a lateral velocity of 1 m/s. It stops on lane 250 after reaching the position 40 meters from the lane's start. The last argument of the function `NPC` lets the vehicle delay its movement 1 second after the ego vehicle becomes ready to move.
- `npc2` is a taxi vehicle with aggressive driving behavior, including sudden stops and high deceleration (9.81 m/s^2). It starts moving simultaneously with the ego vehicle.
- `npc3` is a stationary hatchback placed on lane 249.
- `pedes` is an elegant-style pedestrian, who goes across the north crosswalk in the figure from right to left with a constant speed of 1.5 m/s. Unlike the NPC vehicles, whose routes are defined through traffic lanes and offset distances, the pedestrian's traveling path is defined through a sequence of waypoints. In this case, the pedestrian will be spawned at point (203, -222.5), heading to point (169, -235) and stopping there.
- `ego` denotes the ego vehicle. Its initial pose is set 10 meters behind `npc1` and its goal is on lane 263, 60

Listing 1. AWSIM-Script: An input script example.

```

1 // Position on lane 226, 20 meters from the start point
2 spawnPos = "TrafficLane.226" at 20;
3 goalPos = "TrafficLane.250" at 40;
4 // The route that NPC will travel
5 route1 = [
6   "TrafficLane.226" max-velocity(10),
7   "TrafficLane.427" max-velocity(8.33),
8   "TrafficLane.249" max-velocity(8.33),
9   // lane change at offset 8m, with longitudinal and lateral
      velocities 8.33 and 1, respectively
10  change-lane(8, 8.33, 1),
11  "TrafficLane.250"
12 ];
13 npc1 = NPC("smallcar", spawnPos, goalPos, route1,
      [delay-move-until-ego-engaged(1)]);
14
15 route2 = [
16   "TrafficLane.248",
17   "TrafficLane.449",
18   "TrafficLane.264" max-velocity(13.889)
19 ];
20 npc2 = NPC("taxi", "TrafficLane.248" at 10,
      "TrafficLane.264", route2,
21   [delay-move-until-ego-move(0), deceleration(9.81),
      aggressive-driving]);
22
23 // A stationary NPC
24 npc3 = NPC("hatchback", "TrafficLane.249" at 40);
25
26 // A pedestrian in elegant style
27 pedes = Pedes("elegant", [203 # -222.5, 169 # -235],
      [speed(1.5)]);
28
29 // Ego vehicle
30 ego = Ego(spawnPos back 10, "TrafficLane.263" at 60,
      [max-velocity(8.33)]);
31
32 run(ego, [npc1, npc2, npc3, pedes]);

```

meters from the lane’s start point. The script also sets the maximum velocity for the ego at 8.33 m/s.

Note that the orientation of each vehicle will be automatically aligned with the direction of the corresponding (one-way) traffic lane.

Language Grammar. We use the ANOther Tool for Language Recognition (ANTLR) v4 framework [21] to define the grammar of AWSIM-Script and to implement the parser component. This is a powerful parser generator framework, widely used to build interpreters, compilers, and tools that require parsing formal languages. The AWSIM-Script’s grammar can be found in the GitHub repository mentioned in Section I.

C. Language features

We summarize the AWSIM-Script’s features for describing the positions of participants, the NPC types, the motion tasks for NPCs, the ego settings, and the scenario creation.

Positions. In AWSIM-Script, positions can be defined flexibly and intuitively. A vehicle’s position is typically represented by a traffic lane ID and an offset distance from the start of that lane (e.g., line 2 in Listing 1). Furthermore, AWSIM-Script enables users to define relative positions, such as moving a vehicle a certain distance “back” or “forward” from another point, or shifting it “left” or “right” to adjacent lanes. For example, A back 10 refers to the position 10 meters behind

the given point A. AWSIM-Script automatically aligns vehicle orientation to the direction of the corresponding traffic lane. This built-in functionality eliminates the need for manually calculating exact 3D coordinates to specify positions (and orientations), simplifying scenario specification.

Vehicle and pedestrian types. AWSIM-Script supports five vehicle types: taxi, hatchback, van, small car, and truck, along with two pedestrian styles: elegant and casual. When creating an NPC vehicle and pedestrian in AWSIM-Script, the user can simply specify the desired model/style by passing the corresponding string (e.g., `smallcar` in line 13 of Listing 1).

Motions for NPCs. Users can configure NPC vehicles to perform various tasks, including following a route, changing lanes, adjusting speed, or making abrupt stops. For instance, an NPC can be directed to follow a predefined sequence of lanes (e.g., `route1` and `route2` in Listing 1). In this context, users can assign desired speed for each lane, enabling vehicles to adjust speed according to the road section. In addition, the `change-lane` command allows NPCs to execute lateral maneuvers, such as changing lane and cut-in, with specified longitudinal and lateral velocities. Note that if neither route nor goal is specified, the vehicle will be treated as stationary.

The motion for a pedestrian can be configured through a sequence of waypoints (line 27 in Listing 1). The pedestrian will start at the first waypoint, heading to the next waypoint until reaching the last one and stop there. Walking speed can be customized for each pedestrian.

Configuration options. AWSIM-Script offers a wide array of configuration options that enable users to fine-tune various aspects of the simulation. For instance, the `acceleration` and `deceleration` commands let users define how quickly NPC vehicles speed up or slow down, which is crucial for simulating realistic driving behaviors like gradual braking or abrupt stops. Moreover, options like `aggressive-driving` simulate reckless behavior, providing a useful mechanism for challenging the ADS operation against sudden deceleration and brake. Additionally, AWSIM-Script supports motion delays through commands like `delay-move-until-ego-engaged`, which synchronize NPC actions with the ego vehicle’s readiness and movement. This delay configuration is useful because it usually takes an amount of time for the ADS planning module to compute the route, plan the trajectory, and prepare the ego vehicle for movement. To prevent the NPC from going too far from the ego, the NPC should not move before that moment. These comprehensive configuration options offer users precise control over the simulation environment.

Ego vehicle. We can set the ego vehicle’s initial pose, goal pose, and maximum velocity within AWSIM-Script (line 30 in Listing 1). Without AWSIM-Script, these settings had to be done through the ADS. Note that we do not specify the route for the ego vehicle since the ADS will generate the planning trajectory for it.

Scenario creation. Once the positions, routes, and behaviors of each traffic participant are defined, the `run` command is used to initiate the scenario. The command includes the specified ego vehicle and the list of NPCs (as shown in line 32 of Listing 1). During the simulation, the ego vehicle and NPCs will operate in parallel and may interact according to their

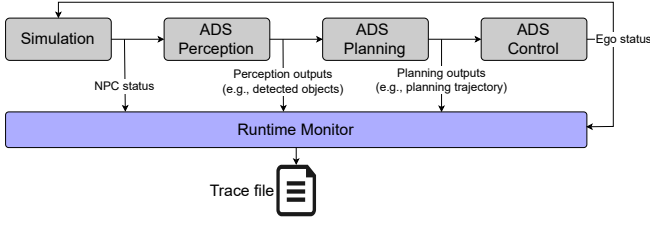


Fig. 5. Data recorded by Runtime Monitor.

predefined behaviors.

AWSIM-Script is not designed to prioritize the ease of creating highly complex scenarios, as many other scenario specification languages do. Instead, its primary focus is on seamless integration with the runtime verification framework, which leverages safety standards to systematically break down driving conditions into a comprehensive set of sufficiently simple scenarios. These scenarios are intentionally neither highly complex nor reliant on the involvement of many traffic actors. Consequently, modeling highly complex scenarios with multiple traffic participants falls outside the scope of this work.

IV. RUNTIME VERIFICATION

This section presents the runtime verification approach used to assess ADS safety. Our framework integrates Runtime Monitor, which captures simulation data, and AW-Checker, which verifies safety properties through LTL model checking.

A. Runtime Monitor

The Runtime Monitor is a core component of our proposed framework designed to capture and log the status of both the ADS and traffic participants during simulation. Its primary role is to gather real-time data from the simulation environment and store this data in a trace file, which can later be used for offline runtime verification (see Figure 5).

In our framework implementation with Autoware and AWSIM-Labs, the monitor was implemented within the simulator, allowing direct access to the statuses of both the ego vehicle and NPC participants. As a result, Runtime Monitor captures data at the same frequency as AWSIM-Labs, which is configured at 40Hz. Most computations in the monitor are lightweight, except for deriving 2D bounding boxes of NPC vehicles and pedestrians on the screen view of ADS's object detection camera. This involves projecting all vertices of the NPC's mesh collider (200–800 vertices) onto the camera screen view, as relying on just 8 vertices of the 3D bounding box often results in oversized 2D bounding boxes. While Unity's default frame rate is 50Hz, we reduced it to 40Hz to account for these additional computations. We carefully observed that during the simulation, the 40Hz is not exceeded, i.e., there is no lag during the simulation.

Data recording. The Runtime Monitor operates continuously during the simulation, collecting detailed data about the ego vehicle, NPC vehicles and pedestrians as well as the outputs from ADS modules. Specifically, the monitor tracks:

- Ego vehicle status: This includes the vehicle's position, orientation, velocity (linear and angular), acceleration,

size, and shape. This data provides a comprehensive snapshot of the ego vehicle's movements.

- NPC status: For each NPC vehicle and pedestrian, the monitor logs the same kind of data as the ego vehicle. Furthermore, if a sensor camera is used for object detection, Runtime Monitor will also capture the NPC's 2D bounding box (position and size) reflected on the camera sensor. This data serves as the ground truth information to evaluate the performance of the camera-based object detection in the perception module later.
- Perception module outputs: The tracked objects from the perception module are recorded. Each tracked object includes information like identifier, existence probability, classification with probability, position, orientation, velocity (linear and angular), acceleration, shape, and 2D bounding box on the camera sensor's screen view. This data is essential for evaluating the accuracy of the ADS's perception with the simulated ground truth.
- Planning module outputs: The planned trajectories generated by the ADS for the ego vehicle are also recorded. These consist of sequences of points, where each point includes the position, orientation, longitudinal velocity, lateral velocity, and acceleration, acting as the planning path for the ego vehicle.

Trace File Generation. At the end of the simulation, the Runtime Monitor dumps the recorded data into a trace file, which serves as a detailed log of the entire scenario. The trace file is configurable and can be output in different formats like YAML or as a Maude state machine representation. The state machine format includes transitions between two consecutive scenes, enabling direct verification of different temporal properties with AW-Checker.

B. AW-Checker

AW-Checker is a crucial component responsible for verifying whether the ADS satisfies the desired safety requirements by checking the generated execution traces. It leverages the Maude LTL model checker to verify various temporal properties, ensuring that the behavior of the ADS adheres to specified safety requirements.

Maude fundamentals

Maude [19] is a high-performance reflective language and system based on rewriting logic, allowing both functional and concurrent system specifications. Maude offers a variety of formal analysis tools, including reachability analysis and LTL model checking, making it ideal for specifying and verifying complex systems. In the following, we briefly introduce some of the core concepts of Maude relevant to AW-Checker.

Modules: Maude's primary unit of specification and programming is the *module*. Maude has two types of modules. *Functional modules* are used to specify equational theories. The core elements are sorts, operators, and equations, which describe data types and functions. *System modules* extend functional modules by adding rewrite rules, which describe dynamic system behavior. They represent concurrent transitions between states, governed by conditions and rules.

Operators and equations: In Maude, operators (or function symbols) are declared with the **op** keyword. They define the functions and constants in the system and specify their input and output sorts. Equations can be unconditional or conditional with the **eq** and **ceq** keywords, respectively. For example, the following Maude code in AW-Checker defines the 3D vector data structure with the addition function:

```
--- FloatOrInt is the sort of float numbers
op _+ : FloatOrInt FloatOrInt FloatOrInt -> Vector3 [ctor] .
op _+_ : Vector3 Vector3 -> Vector3 [assoc comm prec 33] .
vars X Y Z X2 Y2 Z2 : FloatOrInt .    --- variable declarations
eq ((X Y Z) + (X2 Y2 Z2)) = (X + X2) (Y + Y2) (Z + Z2) .
```

Rewrite rules: Rewrite rules in system modules specify state transitions and allow dynamic behaviors to be modeled. Rewrite rules are non-deterministic and describe how the system evolves.

LTL model checking: Maude supports LTL to verify system properties over time. LTL uses temporal operators like: $\Box$ (always $\Box$)—the LTL formula holds at all future points, $\Diamond$ (eventually $\Diamond$)—the formula holds at some point in the future, and $\circ$ (next)—the formula holds at the next state. To model check an LTL formula p , we use the following command:

```
red in MOD : modelCheck(init, p) .
```

Here, the **red** keyword stands for *reduce* (used to simplify or evaluate a term by applying the defined equations and rewrite rules until no further reductions can be applied), **MOD** is the Maude module which contains the state machine definition, and **init** is the initial state of the system.

Sample properties

AW-Checker can verify a variety of properties. A few representative examples are illustrated in Listing 2.

Standard properties. The command at line 2 verifies that no collision occurs between the ego vehicle and the NPC vehicle **npc1**, where $\sim$ and **init** denote the negation $\neg$ and the initial state of the system for the model checker to explore (the initial scene), respectively. Here, **collision** is an atomic proposition whose syntax and semantics are defined by ours (which will be presented below). The second argument, "**npc1**", can be omitted. In that case, the assertion will change to checking there is no collision between the ego vehicle and any NPCs.

The command at line 3 asserts that the distance (position difference) between the two center points of the ego vehicle and the NPC vehicle **npc1** is never less than 5 meters. Line 4 checks that the Time-to-Collision (TTC) with **npc1** is never less than 2 seconds. TTC defines how long it will take for the ego vehicle to collide with the NPC vehicle if they keep going at the same velocities. AW-Checker takes vehicles' bounds, orientations, and velocities (longitude and latitude) into account for computing this TTC value. The two commands at lines 5–8 use combinations of two propositions. The former asserts that when the TTC with **npc1** is under 2 seconds, the ego vehicle's acceleration should drop below -1. The latter checks that when the distance between the ego vehicle and **npc1** is under 20 meters, the ego's speed is less than 3.33 m/s. The command at lines 9–10 asserts that the ego vehicle eventually reaches its destination.

Listing 2. An example of desired safety properties specified in AW-Checker.

```
1  --- General properties
2  red modelCheck(init, [] ~collision("ego", "npc1")) .
3  red modelCheck(init, [] pos-diff("ego", "npc1") >= 5) .
4  red modelCheck(init, [] ttc("npc1") >= 2) .
5  red modelCheck(init, []
6    (ttc("npc1") < 2 -> accelScalar("ego") < -1)) .
7  red modelCheck(init, []
8    (pos-diff("ego", "npc1") < 20 -> speed("ego") < 3.33)) .
9  red modelCheck(init,
10    <> distance("ego", "TrafficLane.263" at 60) < 0.2) .
11
12  --- Perception-related properties
13  red modelCheck(init, [] (pos-diff("ego", "npc1") <= 20
14    -> perp-pos-diff("npc1") < 0.5 within 200 ms)) .
15  red modelCheck(init, [] (pos-diff("ego", "npc1") <= 20
16    -> perp-rota-diff("npc1") < 8 within 200 ms)) .
17  red modelCheck(init, [] (pos-diff("ego", "npc1") <= 20
18    -> (perp-shapelength-diff("npc1") < 0.6 /\
19      perp-shapewidth-diff("npc1") < 0.3) within 200 ms)) .
20  red modelCheck(init, [] (pos-diff("ego", "npc1") <= 40
21    -> IOU-ratio("npc1") >= 0.6 within 350 ms)) .
```

Perception-related properties. In addition to those standard properties above, AW-Checker can verify perception-related properties that assess the accuracy of the ADS's perception module. In Listing 2, the command at lines 13–14 verifies that when **npc1** is less than 20 meters away, the ADS can always perceive it with less than 0.5 meters of distance error within 200 milliseconds.

The command at lines 15–16 checks the orientation perception accuracy. Here, for each NPC vehicle, the orientation checking only considers the rotation around the yaw axis (the vertical axis, running up and down). The rotation unit is degree. Similarly, we can check the perceived shape accuracy (lines 17–19).

When a sensor camera is used for object detection in the perception system, we can also evaluate the accuracy of that camera-based object detection system. The last command concerns the Intersection over Union (IoU) ratio between the 2D detected bounding box and the ground truth counterpart of **npc1** reflected on the camera screen view.

Syntax and semantics implementation

Safety properties for ADSs can be formalized as LTL formulas. These formulas combine atomic propositions (e.g., **collision** in the example above), boolean connectives (e.g., $\neg$), and temporal operators (e.g., $\Box$). AW-Checker leverages this combination to specify and verify properties without the need to introduce new temporal operators.

In AW-Checker, we define some common atomic propositions, which describe logical statements about the system's behavior at any given point in the simulation. Examples include facts about vehicle collisions, relative positions, or TTC thresholds. Users can utilize one or more of these predefined propositions to specify the desired property for verification, such as those shown in Listing 2.

Syntax definition. Maude facilitates the definition of user-friendly syntax for these atomic propositions. The following shows the syntax declaration of some representative atomic propositions predefined in AW-Checker:

```

ops < > <= >= == : -> ComparisonOp [ctor] .
op ____ : ComparableExp ComparisonOp FloatOrInt -> Prop .
op collision : String String -> Prop .
op pos-diff : String String -> ComparableExp .
op ttc : String -> ComparableExp .
op speed : String -> ComparableExp .
op accelScalar : String -> ComparableExp .
op perp-pos-diff : String -> ComparableExp .
op perp-rota-diff : String -> ComparableExp .
op perp-speed-diff : String -> ComparableExp .
op perp-shapelength-diff : String -> ComparableExp .
op perp-shapewidth-diff : String -> ComparableExp .
op IOU-ratio : String -> ComparableExp .
op timeStamp : -> ComparableExp .
op thw : String -> ComparableExp .

```

In this declaration, Prop is the sort of atomic propositions, while ComparableExp and ComparisonOp are two intermediate sorts internally used for implementation. Terms of ComparableExp sort combined with relational operators like $\geq$, $\leq$, $>$, and $<$ form a proposition that asserts comparisons over values, such as distances, speeds, and time to collisions. Most of these propositions have already been introduced in Listing 2. In addition, speed relates to the speed of a given vehicle, while thw relates to the Time-Headway (THW), i.e., the amount of time it takes for the ego vehicle to reach a leading vehicle's position if both maintain their current speeds.

While the four relational operators are infix, the remaining ones are prefix. The flexibility of Maude's mixfix syntax (supporting combinations of prefix, infix, and postfix operators) allows for intuitive proposition definitions. For example, Maude's infix operators simplify comparisons over numerical values, such as checking TTC thresholds or relative distances. **Semantics definition.** To implement the semantics of these atomic propositions, we use Maude as a functional programming language. Let us elaborate on how we define the semantics of the proposition pos-diff. We introduce the predicate pos-diff, which takes a state (a scene) and two participant names as inputs and returns the difference in their positions in the state:

```

-- Maybe{Vector3} is the sort of Vector3 and null
op pos-diff : AWState String String -> Maybe{Vector3} .
-- for state AWST, two strings STR,STR2, and two 3D points P,P2
ceq pos-diff(AWST, STR, STR2) = (P - P2)
  if P := gtCenterPosition(AWST,STR) /\
    P2 := gtCenterPosition(AWST,STR2) .
eq pos-diff(AWST, STR, STR2) = nullVector3 [owise] .

```

In the first (conditional) equation, gtCenterPosition extracts the center points of the two participants's shapes from the state (P and P2). The owise (otherwise) attribute attached to the last equation forces the second equation to be applied only if the first equation cannot be applied. This happens when one of the given participant names (STR and STR2) does not exist.

Using this state predicate, the proposition semantics can be simply defined as follows:

```

ceq AWST |= pos-diff(STR, STR2) < X = true
  if P := pos-diff(AWST, STR, STR2) /\ magnitude(P) < X .

```

Regarding perception accuracy, there is often a slight delay between when an object starts appearing in the ADS tracking region and when the ADS can detect it. AW-Checker accommodates such perception delays through the `_within_ms` operator, which allows the verification of properties while

accounting for a specified delay threshold (in milliseconds). For any LTL formula F , AW-Checker translates term F within X ms to the following LTL formula:

$$F \vee \bigcirc F \vee \dots \vee \underbrace{\bigcirc \dots \bigcirc F}_{k\text{-times}}$$

where $\bigcirc$ is the LTL next temporal operator and $k = X/h$ with h is the monitor capturing frequency in millisecond units (the time interval between two consecutive captured scenes).

Our runtime verification approach utilizes the Maude LTL model checker instead of developing a custom search or checking algorithm. This allows us to leverage the high performance of a well-established model checker, enabling the verification of sophisticated properties in a highly efficient manner.

V. EXPERIMENTS WITH TRAFFIC DISTURBANCE SCENARIOS

This section reports the experimental results that we have conducted with Autoware based on the three traffic disturbance scenarios outlined in the JAMA standard [12]: cut-in, cut-out, and deceleration scenarios. These scenarios represent a combination of non-intersection road geometry and the lane-keeping behavior of the ego vehicle (see Figure 2).

While this study focuses on the three scenarios mentioned above, our framework is capable of simulating all possible traffic disturbances caused by surrounding vehicles, as specified in the JAMA safety standard, and evaluating ADS performance under these disturbances. These three disturbances are among the most dangerous and frequently encountered in real-world driving conditions, requiring the ADS to respond effectively to avoid collisions. Nevertheless, achieving comprehensive coverage of the full range of traffic disturbances outlined in the JAMA standard remains our ultimate goal. To this end, further analysis and experiments involving other traffic disturbances, such as intersection-related scenarios, are left for future work.

The July 2024 release of Autoware was used in the experiments. Each experiment was conducted under two perception modes:

- **Lidar-only:** This is the default mode in Autoware and AWSIM-Labs. The ego vehicle is equipped with three lidars for localization and obstacle detection, along with a camera used exclusively for traffic light detection.
- **Camera-lidar fusion:** In addition to the three lidars and the camera used for traffic light detection, an additional camera is used for object detection, such as recognizing NPC vehicles and pedestrians. Fusion algorithms process the combined sensor data.

The experiments aim to answer four key research questions:

- RQ1:** Does Autoware meet the minimum safety requirements as defined by the JAMA standard via the careful driver model?
- RQ2:** If the desired safety level is not met, where does the main root cause come from?
- RQ3:** Which factors increase the risk for Autoware in these traffic disturbance scenarios?
- RQ4:** Which perception mode, lidar-only or camera-lidar fusion, is more effective for Autoware? (This question arose naturally after observing that Autoware does not perform well in preventing danger during the experiments conducted in the default lidar-only perception mode.)

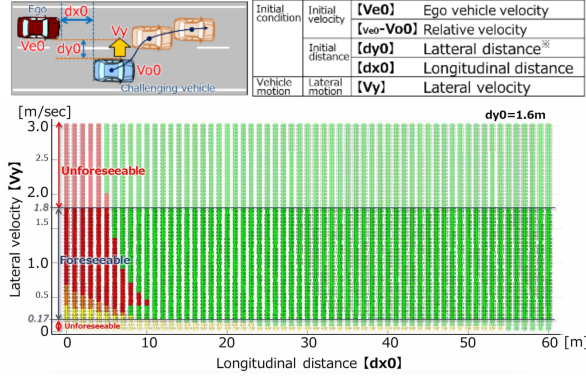


Fig. 6. Cut-in scenario and collision-preventable boundary when ego's and NPC's speeds are 30 and 20 km/h, respectively [12].

A. Cut-in scenario

In a cut-in scenario, a vehicle traveling in the adjacent lane abruptly cuts in front of the ego vehicle. The top of Figure 6, which is borrowed from [12, Section 2.3.3.1.4], visualizes this scenario. The diagram at the bottom of the figure shows the collision-preventable boundary if the ego vehicle is controlled by a careful driver when the ego vehicle and the NPC speeds are 30 and 20 km/h, respectively. The blue, red, orange, and yellow points denote different outcomes: no collisions, front-back collisions, side collisions, and backward interruptions.

For example, when the initial longitudinal distance between the ego vehicle's front and the NPC's rear is 5 meters and the NPC's lateral velocity is 1 m/s, a front-back collision occurs. However, if the same lateral velocity is applied and the longitudinal distance is increased to 10 meters, no collision takes place.

Parameter configuration. Multiple experiments have been conducted using our framework to evaluate Autoware's performance compared to the careful driver model. To this end, some default configurations needed to be adjusted before running the experiments. By default, Autoware's deceleration and jerk configurations for the ego vehicle are set to 2.5 m/s^2 and 1.5 m/s^3 , respectively. These values are insufficient to prevent collisions in scenarios where other vehicles stop suddenly or cut in. We increased these settings to 8.33 m/s^2 ($0.85G$) and 83.3 m/s^3 , which match the capability of Automated Emergency Braking (AEB) system [12, Section 2.3.3.1], and beyond the capability of well-trained human drivers according to [12], [22], [23] (maximum deceleration and jerk of well-trained drivers are 7.59 m/s^2 and 12.65 m/s^3). These settings equip Autoware with emergency reaction capabilities equivalent to the AEB system and superior to human drivers, ensuring that any weaknesses identified later are valid and not caused by insufficient configurations. We also raised the ego vehicle's maximum velocity to 17 m/s, as the default of 4.17 m/s was insufficient for the experiment's conditions. All of the experiments were conducted using the NishiShinjuku map provided by AWSIM-Labs.

Scenario specifications with AWSIM-Script. AWSIM-Script allows for easy specification of cut-in scenarios. Below is an example of a cut-in simulation script:

TABLE I
CUT-IN EXPERIMENTS IN LIDAR-ONLY PERCEPTION MODE.

Config Ve	NPC Vo	ID	NPC Vy	Config dx0	Expected collision	Actual Ve	Actual dx0	Collision	Min TTC
5.56 (20 km/h)	2.78 (10 km/h)	20-10-1	0.6	9	N	5.55	9.30	N	3.91
		20-10-2	0.8	8	N	5.55	8.35	N	2.84
		20-10-3	1.0	8	N	5.55	8.26	N	2.59
		20-10-4	1.2	7	N	5.55	7.32	N	1.65
		20-10-5	1.4	7	N	5.55	7.24	N	1.64
		20-10-6	1.6	6	N	5.55	6.28	N	1.52
8.33 (30 km/h)	5.56 (20 km/h)	30-20-1	0.6	10	N	8.33	10.16	N	3.50
		30-20-2	0.8	8	N	8.28	8.16	N	2.13
		30-20-3	1.0	7	N	8.32	7.18	N	1.52
		30-20-4	1.2	7	N	8.26	7.12	N	1.59
		30-20-5	1.4	6	N	8.28	6.14	N	1.13
		30-20-6	1.6	6	N	8.31	6.09	N	0.78
8.33 (30 km/h)	2.78 (10 km/h)	30-10-1	0.6	16	N	8.34	16.60	N	2.29
		30-10-2	0.8	15	N	8.33	15.64	N	1.73
		30-10-3	1.0	14	N	8.33	14.53	N	1.26
		30-10-4	1.2	12	N	8.33	12.56	N	0.61
		30-10-5	1.4	12	N	8.34	12.61	N	0.72
		30-10-6	1.6	11	N	8.33	11.64	Y	
11.11 (40 km/h)	8.33 (30 km/h)	40-30-1	0.6	10	N	11.11	9.97	N	2.85
		40-30-2	0.8	8	N	11.11	8.04	N	2.18
		40-30-3	1.0	7	N	11.11	7.16	N	1.62
		40-30-4	1.2	7	N	11.11	6.99	N	1.56
		40-30-5	1.4	6	N	11.11	6.08	N	1.23
		40-30-6	1.6	6	N	11.11	6.13	N	1.34
11.11 (40 km/h)	5.56 (20 km/h)	40-20-1	0.6	19	N	11.12	19.19	N	2.89
		40-20-2	0.8	16	N	11.12	16.24	N	1.54
		40-20-3	1.0	14	N	11.11	14.19	N	0.86
		40-20-4	1.2	13	N	11.10	13.18	Y	
		40-20-5	1.4	12	N	11.12	12.11	Y	
		40-20-6	1.6	12	N	11.12	12.01	Y	
13.89 (50 km/h)	11.11 (40 km/h)	50-40-1	0.6	10	N	13.78	10.11	N	3.75
		50-40-2	0.8	8	N	13.83	8.24	N	2.15
		50-40-3	1.0	7	N	13.79	7.19	N	2.05
		50-40-4	1.2	7	N	13.77	7.31	N	2.05
		50-40-5	1.4	6	N	13.76	6.22	N	1.63
		50-40-6	1.6	6	N	13.75	6.03	N	1.65
13.89 (50 km/h)	8.33 (30 km/h)	50-30-1	0.6	19	N	13.83	19.24	N	2.65
		50-30-2	0.8	16	N	13.84	16.22	N	1.48
		50-30-3	1.0	14	N	13.84	14.16	N	0.86
		50-30-4	1.2	13	N	13.83	13.22	Y	
		50-30-5	1.4	12	N	13.84	12.20	Y	
		50-30-6	1.6	12	N	13.84	12.24	Y	

```

spawnPos = "TrafficLane.112";
goal = "TrafficLane.111" at 220;
npcspeed = 5.556;
route1 = [
  "TrafficLane.112" max-velocity(npcspeed),
  cut-in(_, npcspeed, 1, 7),
  "TrafficLane.111" max-velocity(npcspeed)
];
npc1 = NPC("hatchback", _, goal, route1, [acceleration(12)]);
ego = Ego(spawnPos left 0, "TrafficLane.111" at 190,
  [max-velocity(8.333)]);
run(ego, [npc1], [saving-timeout(90)]);

```

Here, traffic lanes 111 and 112 are two adjacent lanes on the map used. The cut-in function takes four parameters as inputs. The first one is the offset (distance from the lane's start point) to start the cut-in. By using the placeholder `_`, we leave this value for AWSIM-Script to compute it automatically. The three remaining parameters are the longitudinal velocity (v_{o0}), the lateral velocity (v_y) of the cut-in, and the longitudinal distance between two vehicles (dx_0) at the cut-in start. Similarly, the NPC function automatically computes the NPC vehicle's spawning position to ensure that the scenario's constraints are satisfied at the cut-in start. The option `saving-timeout(90)` attached to the `run` command sets the timeout for the simulation, that is, the trace will be exported after 90 seconds or when the ego vehicle reaches its goal, whichever comes first. We set a high acceleration value for the NPC to make it reach the desired speed quickly.

Experimental results. The results from the cut-in experiments, performed in lidar-only and camera-lidar fusion modes,

TABLE II
CUT-IN EXPERIMENTS IN CAMERA-LIDAR FUSION PERCEPTION MODE.

Config Ve	NPC Vo	ID	NPC Vy	Config dx0	Expected collision	Actual Ve	Actual dx0	Collision	Min TTC
5.56 (20 km/h)	2.78 (10 km/h)	20-10-1	0.6	9	N	5.55	9.27	N	3.59
		20-10-2	0.8	8	N	5.55	8.23	N	1.83
		20-10-3	1.0	8	N	5.55	8.34	N	2.18
		20-10-4	1.2	7	N	5.55	7.26	N	1.46
		20-10-5	1.4	7	N	5.55	7.31	N	1.70
		20-10-6	1.6	6	N	5.55	6.36	N	1.25
8.33 (30 km/h)	5.56 (20 km/h)	30-20-1	0.6	10	N	8.33	10.11	N	1.79
		30-20-2	0.8	8	N	8.27	8.07	N	1.35
		30-20-3	1.0	7	N	8.31	7.22	N	1.16
		30-20-4	1.2	7	N	8.29	7.12	N	1.23
		30-20-5	1.4	6	N	8.26	6.25	N	0.93
		30-20-6	1.6	6	N	8.30	6.05	N	1.00
8.33 (30 km/h)	2.78 (10 km/h)	30-10-1	0.6	16	N	8.33	16.49	N	1.79
		30-10-2	0.8	15	N	8.33	15.64	N	0.73
		30-10-3	1.0	14	N	8.33	14.67	N	1.08
		30-10-4	1.2	12	N	8.34	12.61	Y	
		30-10-5	1.4	12	N	8.34	12.57	Y	
		30-10-6	1.6	11	N	8.33	11.67	Y	
11.11 (40 km/h)	8.33 (30 km/h)	40-30-1	0.6	10	N	11.12	10.01	N	2.72
		40-30-2	0.8	8	N	11.11	8.02	N	1.66
		40-30-3	1.0	7	N	11.12	7.04	N	1.56
		40-30-4	1.2	7	N	11.11	7.10	N	1.63
		40-30-5	1.4	6	N	11.11	6.01	N	1.13
		40-30-6	1.6	6	N	11.11	6.15	N	1.14
11.11 (40 km/h)	5.56 (20 km/h)	40-20-1	0.6	19	N	11.12	19.30	N	1.91
		40-20-2	0.8	16	N	11.12	16.14	N	1.48
		40-20-3	1.0	14	N	11.12	14.11	Y	
		40-20-4	1.2	13	N	11.12	13.34	Y	
		40-20-5	1.4	12	N	11.12	12.11	Y	
		40-20-6	1.6	12	N	11.12	12.35	Y	
13.89 (50 km/h)	11.11 (40 km/h)	50-40-1	0.6	10	N	13.82	10.00	N	3.24
		50-40-2	0.8	8	N	13.83	8.16	N	2.10
		50-40-3	1.0	7	N	13.80	7.21	N	1.76
		50-40-4	1.2	7	N	13.79	7.24	N	1.92
		50-40-5	1.4	6	N	13.80	6.28	N	1.35
		50-40-6	1.6	6	N	13.79	6.04	N	1.33
13.89 (50 km/h)	8.33 (30 km/h)	50-30-1	0.6	19	N	13.84	19.22	N	2.25
		50-30-2	0.8	16	N	13.84	16.24	N	1.54
		50-30-3	1.0	14	N	13.84	14.19	Y	
		50-30-4	1.2	13	N	13.83	13.13	Y	
		50-30-5	1.4	12	N	13.84	12.22	Y	
		50-30-6	1.6	12	N	13.84	12.37	Y	

are shown in Tables I and II, respectively. The first and second columns denote the configured velocities of the ego and cut-in vehicles, respectively. The actual ego vehicle's velocity at the moment when the cut-in starts may be different from the configured value since we cannot control such actual value (for example, the ego may reduce the speed when approaching the NPC vehicle). The Actual Ve column records this actual velocity. The third column denotes the experiment ID. The fourth and fifth columns denote the lateral velocity of the cut-in and the configured longitudinal distance when the cut-in starts, respectively. Similarly to the ego's velocity, the actual longitudinal distance, which is recorded in the eighth column, may be slightly different from the configured value (due to the uncontrollable ego velocity). The Expected collision column indicates whether collisions were expected based on the careful driver model, while whether collisions actually occurred in our experiments is recorded in the Collision column. In non-collision cases, the last column records the minimal TTC between the ego and cut-in vehicles, providing an indication of how close the vehicles came to colliding.

The experiments were conducted with a lateral velocity for the cut-in ranging from 0.6 to 1.6 m/s. According to statistic data [12, Section 2.3.3.1.1], the average lateral velocity is 1.0 m/s, with a maximum value of 1.8 m/s, and it rarely falls below 0.5 m/s. This ensures that the selected range reflects realistic lateral velocities observed in actual driving scenarios.

Collision/non-collision checking can be done by the model checking command as introduced in Section IV-B. Besides,

AW-Checker also supports retrieving (1) the ego's actual speed and the actual distance dx0 at the start of the cut-in maneuver for compliance checking, and (2) the minimal TTC to assess how close the ego vehicle came to colliding with the challenging vehicle.

The experiments revealed several key insights regarding Autoware's performance in cut-in scenarios and the comparison with the careful driver model.

Collision avoidance is not as good as the careful driver model. While the careful driver never makes a collision, Autoware did fail to avoid collisions in several scenarios, particularly in high-risk cases with high relative speeds. For example, in experiment 30-10-6, in which the ego vehicle was traveling at 30 km/h and the NPC at 10 km/h, a collision occurred (marked as "Y") in both lidar-only and camera-lidar fusion perception modes. This result, in contrast to the careful driver model, highlights a significant limitation in Autoware's decision-making and reaction times when faced with sudden lane changes by vehicles traveling on adjacent lanes.

RQ1 was answered in these cut-in experiments: Autoware does not meet the minimum safety requirement defined via the careful human driver model.

Errors in travel path prediction are the root cause of collisions. We observed that in each cut-in experiment, the Autoware perception module consistently predicted the travel paths of the cut-in vehicle as slow and gradual. However, in cases where the cut-in vehicle had a high lateral velocity, such as 1.6 m/s, the lane change was much more aggressive than Autoware's predictions. This discrepancy between the predicted gentle lane change and the actual fast maneuver led to delayed braking by the ego vehicle, which is the root cause of the collisions observed in these experiments. Autoware's assumption that lane changes occur gently, rather than accounting for more abrupt movements, hindered its ability to respond properly and safely to these high-velocity cut-ins. This explanation answers **RQ2**.

Risky at high speeds. In terms of collision avoidance, Autoware's performance was significantly worse at higher speeds. For example, in the lidar-only mode, with the same relative speed of 20 km/h, collisions occurred in only one experiment when the ego's speed was 30 km/h (ID 30-10-6), while there were three collision cases when the ego's speed was 50 km/h (IDs 50-30-4, 50-30-5, and 50-30-6). In terms of TTC, with the same relative speed, the minimum TTC is often lower with lower speeds of the ego and the cut-in vehicles (e.g., cases 20-10-* have greater TTCs than cases 40-30-*).

Lateral velocity also had a significant impact on collision avoidance capability. From the two tables, it is easy to see that higher lateral velocities increased the likelihood of collisions in both perception modes. For instance, in both lidar-only and camera-lidar fusion modes, when the velocities of the ego and the cut-in vehicles were 40 and 20, respectively, Autoware could prevent collisions if the NPC's lateral velocity was under 1 m/s but failed to do so with higher lateral velocities. With the same longitudinal velocities, the higher the lateral velocity the lower the TTC.

These observations suggest that Autoware struggles with high speeds and fast lateral movements by other vehicles,

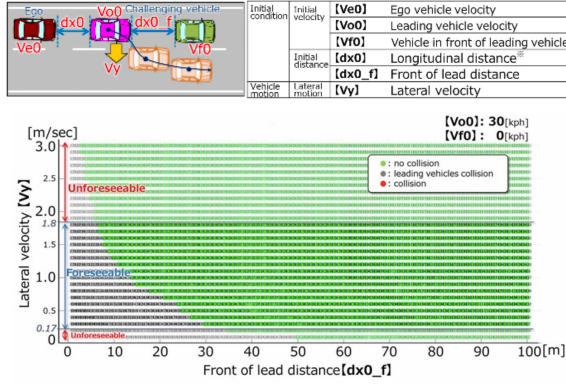


Fig. 7. Cut-out scenario and collision-preventable boundary when ego's speed is 30 km/h [12].

answering **RQ3**.

Lidar-only is more effective than Camera-lidar fusion.

Another important observation from the experimental results is that the camera-lidar fusion mode generally performed worse than the lidar-only mode, especially in situations involving high relative speeds and lateral velocities. This performance disparity between the two modes is evident in the collision rates and minimal TTC values. For example, in Experiments 30-10-4 and 30-10-5, collisions occurred in the camera-lidar fusion mode, while in the lidar-only mode, Autoware could prevent collisions. For each non-collision experiment, the TTC value in the lidar-only mode is often higher than that in the camera-lidar fusion mode.

Theoretically, the fusion of camera and lidar data should provide richer perception information. The experimental results, however, revealed an opposite outcome. Analyzing the traces with AW-Checker, we found that the machine learning-based object detection algorithm was likely a primary factor in the underperformance of the camera-lidar perception mode. For instance, in experiment 30-10-4, 71% of cases had an IOU ratio below 0.3, while only 16.67% exceeded 0.5. These inaccuracies could act as noise to the fusion algorithm, resulting in imprecise NPC vehicle detection. We also guess that additional timing delays introduced by the fusing algorithm when processing data from two different sensor types could be another potential factor contributing to the underperformance.

Demonstration of the collisions and other materials supporting the experiments are available at <https://github.com/duongtd23/AW-Runtime-Verification>.

B. Cut-out scenario

In a cut-out scenario, a vehicle ahead traveling on the same lane as the ego vehicle suddenly changes its lane to the adjacent lane to avoid a decelerating or stopped vehicle (a broken-down car or due to a traffic jam). This forces the ego vehicle to either decelerate or change lanes to avoid a collision. The first fatal Tesla Autopilot crash¹ was caused by a cut-out scenario. Therefore, it is crucial for Autoware to be able to avoid danger in this scenario.

Figure 7 visualizes this scenario and shows the collision-preventable boundary according to the careful driver model when the ego and the cut-out vehicles are traveling at 30 km/h. The red, pink, and green cars are the ego vehicle, the cut-out vehicle, and the challenging vehicle, which the ego should avoid collision with. Initially, the ego and the cut-out vehicles are traveling at the same velocity (v_{e0} and v_{o0}), with a longitudinal distance (dx_0) of twice the THW between them ($THW = 2$ seconds). For instance, when their speeds are 30 km/h, the initial distance is 16.67 meters.

In practice, achieving the 2-second THW constraint in Autoware is challenging, as Autoware often maintains a greater distance (greater THW) to the vehicle ahead for safety. As a solution, we propose to dynamically spawn the challenging vehicle at the moment the cut-out starts, ensuring that the distance from the spawning position to the ego vehicle is equal to the sum of the two desired distances dx_0 and dx_{0-f} . Subsequently, a cut-out scenario can be specified like the following script:

```
spawnPos = "TrafficLane.112";
goal = "TrafficLane.111" at 220;
speed = 5.555;
route1 = [
    "TrafficLane.112" max-velocity(speed),
    cut-out(150, speed, 1, 10),
    "TrafficLane.111" max-velocity(speed)
];
npc1 = NPC("hatchback", spawnPos forward 15, goal, route1,
    [acceleration(1), delay-move-until-ego-engaged(1.5)]);
npc2 = NPC("hatchback", _, [delay-spawn(_)]);
ego = Ego(spawnPos, "TrafficLane.112" at 190,
    [max-velocity(speed)]);
run(ego, [npc1, npc2], [saving-timeout(50)]);
```

Here, npc1 and npc2 are the cut-out and challenging vehicles, respectively. The four parameters in the cut-out function respectively specify the offset at which the cut-out starts, the longitudinal velocity, the lateral velocity of the cut-out, and the distance dx_{0-f} . AWSIM-Script automatically calculates the appropriate spawning position as well as the spawning moment for npc2 (two placeholders in the NPC function) to ensure the desired distance constraint dx_{0-f} .

Although dynamically spawning the challenging vehicle can only happen in the simulation world, similar situations can occur in reality, where the ego vehicle's vision may be temporarily blocked. For instance, this can happen when a passing car splashes water on the windshield in rainy conditions or when a foreign object temporarily flies into the front of the windshield in windy weather.

Experiment setup. We conducted a series of cut-out experiments with longitudinal velocities of 20, 30, 40, and 50 km/h. For the lateral velocity of the cut-out vehicle, we selected two values: 1.6 m/s and 1.4 m/s. These velocities were chosen because they present the highest likelihood of collisions—the faster the lateral velocity, the closer the challenging vehicle can be placed to the ego vehicle without colliding during the cut-out maneuver. In particular, if Autoware can prevent a collision at a lateral velocity of 1.4 m/s, it should be capable of doing so at lower lateral velocities as well. The same configuration and parameters used for Autoware and AWSIM-Labs in the cut-in experiments were applied in these cut-out experiments.

¹<https://www.nytimes.com/2016/09/15/business/fatal-tesla-crash-in-china-involved-autopilot-government-tv-says.html>

TABLE III
EXPERIMENTS WITH CUT-OUT SCENARIOS.

Mode	ID	Config Ve, Vo	NPC Vy	Config dx0_f	Expected collision	Actual Ve	Collision	Min TTC
Lidar	20-1	5.56	1.6	5	N	5.55	N	2.61
	20-2	5.56	1.4	6	N	5.56	N	2.74
	30-1	8.33	1.6	8	N	8.31	N	1.59
	30-2	8.33	1.4	9	N	8.34	N	1.38
	40-1	11.11	1.6	11	N	11.12	N	1.89
	40-2	11.11	1.4	13	N	11.12	N	1.88
	50-1	13.89	1.6	14	N	13.71	Y	
	50-2	13.89	1.4	17	N	13.71	Y	
Camera-lidar fusion	20-1	5.56	1.6	5	N	5.56	N	2.19
	20-2	5.56	1.4	6	N	5.55	N	2.48
	30-1	8.33	1.6	8	N	8.32	N	1.25
	30-2	8.33	1.4	9	N	8.29	N	2.09
	40-1	11.11	1.6	11	N	11.12	N	1.08
	40-2	11.11	1.4	13	N	11.12	N	1.41
	50-1	13.89	1.6	14	N	13.72	Y	
	50-2	13.89	1.4	17	N	13.71	Y	

Experimental results. Table III shows the experimental results. According to the careful driver model, no collisions should occur in any of these experiments. However, Autware was unable to prevent collisions when the longitudinal velocity was 50 km/h, under both lidar-only and camera-lidar fusion modes. This indicates that the minimum safety level defined by the JAMA standard via the careful driver model is again not met in the cut-out scenario. Two key findings observed in the cut-in experiments were also evident in these cut-out experiments. First, Autware struggled to maintain safety at higher speeds. Second, the camera-lidar fusion mode was less effective than the lidar-only mode, as evidenced by the TTC values (except in experiment 30-2, TTC values in the lidar-only mode are greater than those in the camera-lidar mode).

Root cause of collisions. To answer RQ2, we used AW-Checker to analyze the traces generated in these cut-out experiments. We found two main factors contributing to the collisions in these cut-out experiments: delays in vehicle detection and insufficient application of deceleration force. For instance, in Experiment 50-1 under the camera-lidar fusion perception mode, Autware could detect the challenging vehicle only when it was approximately 19 meters away, even though the initial distance between them was around 45 meters. Furthermore, after detecting the vehicle, Autware failed to apply full deceleration. The maximum deceleration recorded in the trace was only around 5.0 m/s². Under these conditions, it requires at least 20 meters for the ego vehicle to come to a complete stop (assuming immediate braking without jerk time). Consequently, a collision was inevitable.

C. Deceleration scenario

In a deceleration scenario, a vehicle ahead traveling on the same lane as the ego vehicle suddenly decelerates and stops at a deceleration of 1.0G. Similar to the cut-out scenario, it requires that the ego vehicle maintains a distance to the NPC at 2 seconds THW and they travel at the same velocity before the deceleration starts. The safety standards for deceleration scenarios require that a collision should never happen.

Similar to the cut-out scenario, we propose to dynamically spawn the challenging vehicle to achieve the 2-second THW

TABLE IV
EXPERIMENTS WITH DECELERATION SCENARIOS.

Perception mode	ID	Config Ve, Vo	Expected collision	Actual Ve	Collision	Min TTC
Lidar	20	5.56	N	5.55	N	1.54
	30	8.33	N	8.33	N	1.59
	40	11.11	N	11.11	Y	
	50	13.89	N	13.77	Y	
Camera-lidar fusion	20	5.56	N	5.55	Y	
	30	8.33	N	8.33	N	1.25
	40	11.11	N	11.12	Y	
	50	13.89	N	13.76	Y	

constraint in Autware. We have conducted experiments with vehicle velocities of 20, 30, 40, and 50km/h.

The experimental results are summarized in Table IV. Once again, collisions happened with Autware when the velocities were 40 and 50 km/h in both perception modes. Additionally, at 20 km/h, a collision occurred in the camera-lidar fusion mode, while the lidar-only mode successfully prevented it. This implies again that adding a camera for object detection does not improve the performance of Autware's perception system. Delay in obstacle detection is the primary cause of the collisions. For example, in Experiment 50, Autware only recognized the challenging vehicle as an obstacle when it was approximately 15 meters away. This late detection made it impossible for the ego vehicle to brake in time and come to a complete stop, resulting in a collision.

Reliability of the experimental results. To demonstrate the reliability of our experimental results reported above, we have also conducted a second set of experiments for the cut-out and deceleration scenarios. The results and discussion can be found at <https://github.com/duongtd23/AW-Runtime-Verification>.

VI. RELATED WORK

ADS testing and verification. If ADSs were tested in real-world environments using traditional methods, estimates [24] suggest that more than 11 billion miles of driving would be needed to achieve 95% confidence that an ADS is 20% safer than a human driver, making generic testing impractical. Given this, scenario-based testing and verification have become essential for ADS safety analysis, as they allow us to focus on challenging scenarios more efficiently. Several studies have focused on scenario-based testing and verification of ADSs.

VerifAI [25] is a software toolkit designed for the analysis and verification of AI-based systems, focusing on those used in autonomous driving. VerifAI primarily focuses on falsification, employing simulation-driven testing to identify failure scenarios. A key component of VerifAI is its integration with Scenic [26], a probabilistic programming language used to specify abstract driving scenarios. These abstract scenarios describe parameters like the positions and behaviors of objects without specifying exact values. Using sampling techniques, VerifAI generates concrete scenarios from these abstract descriptions, which can then be run as test cases for the ADS. However, the quality and coverage of the generated scenarios depend heavily on how well the abstract scenarios are defined and how parameters are chosen.

TABLE V
SUMMARIZATION OF EXISTING STUDIES.

Study	Main focus	SUT	Simulator	Input
[7]	Critical	Apollo	SVL	Abstract scenarios
[4]	Critical	Interfuser TCP, LAV	CARLA	Abstract models
[5], [9] [10], [11]	Critical	Apollo	SVL	Initial test cases
[17]	Route planning	Apollo	SVL	Initial test cases
[6]	Critical	Lane Follower [6]	CARLA	Abstract scenarios
[28]	Critical	CMCDOT [29]	CARLA	Abstract models
[8]	Critical	NA	Unclear	Abstract models
[30]	Critical	NA	BeamNG [31]	Accident text reports
[32]	Critical	NA	Webots [33]	Dashcam videos
[34]	Critical	NA	BeamNG [31]	Accident visual reports

Our study: Autoware’s safety is evaluated relying on the JAMA standard.

*SUT, Critical, and NA stand for “System under test”, “Critical scenario/test case generation”, and “Not applicable”, respectively.

Using the VerifAI toolkit, Viswanadha et al. [7] have tested Baidu Apollo [16] with the SVL simulator [2], focusing on identifying failure scenarios. Scenic was employed to define abstract scenarios that capture high-level spatial and behavioral relationships within the driving environment. After generating concrete scenarios by sampling techniques, VerifAI was employed to search for failure cases during the simulation. Several critical issues were found, such as failures in intersection handling and pedestrian avoidance. However, the study primarily considered system-level properties, such as non-collision and maintaining a minimum distance from other vehicles. AW-Checker is capable of verifying component-level properties as well.

The VIVAS framework [27] uses symbolic model checking combined with system-level simulation to verify the safety of AI-based autonomous systems. It requires formal models of the environment and the ADS to generate abstract scenarios using model-checking techniques. These abstract scenarios are then instantiated with concrete parameters to produce concrete scenarios that can be executed in the CARLA simulator [1]. However, there are mismatches between the symbolic model used in the framework and the concrete behaviors in the CARLA simulation environment. Despite generating abstract scenarios, not all of them are fully realized in the simulation.

Using the VIVAS framework, Goyal et al. [4] have conducted experiments to analyze the safety of some AI-based perception solutions for autonomous vehicles, including Interfuser [35], TCP [36], and LAV [37], on highway roads. However, all of these AI systems impose a significant speed limitation on the ego vehicle, restricting it to just 5 m/s—well below the capabilities of ADSs like Autoware. As demonstrated in the experiments, higher speeds often cause Autoware to struggle with maintaining safety, while no collisions occur at lower speeds, such as 5.56 m/s.

There are numerous studies on automated testing of ADS and Advanced Driver Assistance Systems (ADAS) in simulated environments, as summarized in Table V. Most of these studies focus on generating or identifying critical scenarios and test cases that are likely to expose system weaknesses. Apart from Apollo, all other Systems Under Test (SUT) in the table either have lower levels of autonomy (e.g., Lane Follower [6],

an ADAS) or are individual ADS modules (e.g., Interfuser [35] and CMCDOT [29], which handle perception).

Among the studies listed in the table, some generate critical scenarios by relying on abstract scenarios or formal models [6], [8], following approaches similar to VIVAS and VerifAI. Some others, such as [5], [9]–[11], expand the set of initial test cases using techniques like fuzzing to uncover critical scenarios. These methods typically fine-tune parameters such as vehicle speed, position, and interaction timing to maximize the likelihood of failure. Formal method techniques can be employed (e.g., [28] used conformance testing) to generate all possible scenarios matching a given situation (e.g., collision or near-miss). However, two key concerns are raised: a huge number of scenarios generated and their realism (e.g., whether an identified collision is indeed due to the SUT’s weakness or because avoiding a collision in the scenario is inherently infeasible). Several other studies utilize real-world data, such as accident reports and dashcam videos, to reconstruct dangerous scenarios [30], [32], [38]. In this line of research, machine learning techniques are often employed to extract detailed information from the data and generate test scenarios consistent with this information.

Unlike all of the studies mentioned above that primarily focus on critical case identification and generation, we base our approach on a well-established safety standard—the JAMA standard [12]—which offers a systematic methodology for evaluating ADS safety. Employing this standard, we conducted experiments focusing on challenging cases to assess Autoware’s performance. While previous studies often define their own safety metrics, we adhere to the careful driver model defined by the JAMA standard as the safety benchmark for analyzing Autoware. Our ultimate goal is to systematically verify Autoware in accordance with the JAMA standard.

Moreover, this study is the first to attempt either verification or automated testing of Autoware. While Autoware and Baidu Apollo [16] are the only open-source platforms supporting full autonomy, numerous studies have focused on safety analyses of Baidu Apollo [5], [9], [10], [17], [18]; to our knowledge, no studies have attempted safety analysis on Autoware.

Scenario specification languages. Table VI summarizes a feature comparison of AWSIM-Script with several other scenario specification languages. Like other languages, AWSIM-Script supports common features, such as defining travel routes, speed profiles, and acceleration profiles for vehicle motion. While other languages emphasize advanced capabilities for creating highly complex scenarios—such as Scenic and SceML, which enable the generation of multiple scenarios from an abstract one—they can be cumbersome to use due to their less user-friendly formats (e.g., the XML format of OpenSCENARIO) or complex, overwhelming syntax (e.g., intricate node and tree definitions in SceML).

In contrast, AWSIM-Script prioritizes precise controllability of traffic participants, ease of implementing scenarios defined by safety standards, and seamless integration with our verification framework. As previously mentioned, AWSIM-Script was designed to be simulator-independent, with its parser component reusable across different simulators. While the current version of AWSIM-Script does not support specifying weather

TABLE VI
FEATURE COMPARISON: AWSIM-SCRIPT VS. OTHER LANGUAGES

Feature		AWSIM-Script	OpenSCENARIO [39]	ScenarioRunner [40]	Scenic [26]	SceML [41]	M-DSL [42]
Format		Script	XML	Python language	Script	Graph	Script
Position	Lane-offset position	✓	✓	✓	✗	✗	✓
	Relative position	✓	✓	✓	✗	✗	✓
Motion	Route/Speed/Acceleration/Deceleration profile	✓	✓	✓	✓	✓	✓
	Trigger events and triggered actions	Partial	✓	✓	✗	✓	✓
Weather		✗	✓	✓	✓	✗	✓
Scenario generation from abstract		✗	✗	✗	✓	✓	✓
Simulator independence		✓	✓	✗	✓	✗	✓
Accurately realize scenarios defined by Safety standard		✓	Partial	✗	✗	✗	✗

conditions, extending its interface to include this element is straightforward. However, enabling this feature in simulators like AWSIM-Labs, which currently lack this capability, would require significant development effort. Additionally, AWSIM-Script has limited support for trigger events and triggered actions, currently restricted to vehicle spawning and initiating movement, compared to other well-known languages like OpenSCENARIO. Improving these capabilities is one piece of our future work, which will be discussed in the last section.

VII. CONCLUDING REMARKS

In this paper, we presented a unified framework for run-time verification of Autoware, an open-source Autonomous Driving System. The framework integrates AWSIM-Script, a scenario specification language; Runtime Monitor, a tool for capturing detailed simulation data; and AW-Checker, an LTL-based verification tool, to facilitate both the simulation of traffic scenarios and the verification of safety requirements. By leveraging the JAMA safety standard, we conducted a series of experiments to assess Autoware’s capability to handle different traffic disturbances. The results demonstrated that while Autoware performed well in some situations, it failed to meet the minimum safety standards in others, particularly in high-speed scenarios. The found weaknesses will be highly beneficial to the Autoware team in improving it, which is crucial as they have deployed several autonomous driving services on public roads². Furthermore, the presented framework will serve as a foundation for conducting further analyses with the other scenarios defined in the JAMA standard.

Framework Generalization. While this paper focuses on implementing our framework with Autoware and AWSIM-Labs, its modular architecture (Figure 3) makes it definitely adaptable to other simulators and ADSs. Among the framework’s modules, AW-Checker is completely independent of simulators and ADSs used, and AWSIM-Script can be partially reused with other simulators (its parser component is reusable, while the simulator adapter would need customization for different simulators). Runtime Monitor would need to be re-implemented for a different simulator, but this should not be overly challenging as its sole responsibility is to record real-time data during simulations for offline verification. However, depending on the ADS under verification, it may be difficult

or even impossible to access the internal status of the ADS for gathering information from its sub-components, such as planning trajectories.

Because Autoware serves as a customizable platform from which various autonomous driving systems can be derived, our presented verification framework is not tied to a single system. In fact, several Autoware-based systems have already been deployed, including taxi and bus services in some cities². Our framework can be utilized to perform safety analyses for these systems.

This paper focuses on Autoware as there is no prior research on testing/verification of it despite the fact that several real driving services powered by Autoware have been deployed. Since the choice of simulator is constrained by the ADS under verification, we chose AWSIM-Labs because Autoware seamlessly supports it. Nevertheless, implementing the framework with other ADSs and simulators is entirely feasible and represents a promising direction for future work. Such implementations would further validate the generalizability and adaptability of our framework to diverse environments.

Scope Limitation. Despite the interesting results obtained from our analysis experiments, their scope is limited to non-intersection road geometries. To comprehensively assess Autoware’s safety, further analyses are required across a broader range of map regions and traffic disturbance conditions, including scenarios involving pedestrians. Evaluating Autoware’s safety in these scenarios is crucial for comprehensively understanding its capabilities and limitations. However, a key challenge in this line of future work is the lack of well-defined safety benchmarks (like the collision-preventable boundary in cut-in and cut-out scenarios) for such scenarios within the JAMA standard. To address this, we plan to derive scenario-specific safety benchmarks based on the JAMA’s careful driver model. Once these benchmarks are established, the runtime verification approach presented in this paper can be applied to conduct further analyses.

Additionally, vehicle disturbances caused by, for example, adverse weather conditions and varying road surface characteristics, should be taken into account in future work. Unfortunately, the simulator used in this study—AWSIM-Labs—does not support these functionalities. AWSIM-Labs, in its current state, only provides the NishiShinjuku city map, which cannot be customized for weather conditions or road surface variations. A possible solution would involve creating

²<https://autoware.org/case-studies/>

new maps, extending the simulator, and updating the AWSIM-Script interface to support additional driving environments, such as highways and rural areas, as well as challenging road conditions, including wet or icy surfaces. However, this process would be highly time-consuming and demand considerable effort.

REFERENCES

- [1] A. Dosovitskiy, G. Ros, F. Codevilla, A. M. López, and V. Koltun, “CARLA: an open urban driving simulator,” in *1st Annual Conference on Robot Learning, CoRL 2017*, vol. 78, pp. 1–16, 2017.
- [2] G. Rong et al., “LGSVL simulator: A high fidelity simulator for autonomous driving,” in *23rd IEEE International Conference on Intelligent Transportation Systems, ITSC 2020*, pp. 1–6, 2020.
- [3] S. Shah, D. Dey, C. Lovett, and A. Kapoor, “Airsim: High-fidelity visual and physical simulation for autonomous vehicles,” in *Field and Service Robotics, Results of the 11th International Conference, FSR 2017*, vol. 5 of *Springer Proceedings in Advanced Robotics*, pp. 621–635, 2017.
- [4] S. Goyal, A. Griggio, J. Kimblad, and S. Tonetta, “Automatic generation of scenarios for system-level simulation-based verification of autonomous driving systems,” in *FMAS@iFM 2023*, vol. 395 of *EPTCS*, pp. 113–129, 2023.
- [5] Y. Zhou, Y. Sun, Y. Tang, Y. Chen, J. Sun, C. M. Poskitt, Y. Liu, and Z. Yang, “Specification-based autonomous driving system testing,” *IEEE Trans. Software Eng.*, vol. 49, no. 6, pp. 3391–3410, 2023.
- [6] Y. Abeysirigoonawardena, F. Shkurti, and G. Dudek, “Generating adversarial driving scenarios in high-fidelity simulators,” in *International Conference on Robotics and Automation, ICRA 2019*, pp. 8271–8277, 2019.
- [7] K. Viswanadha, F. Indaheng, J. Wong, E. Kim, E. Kalvan, Y. Pant, D. J. Fremont, and S. A. Seshia, “Addressing the IEEE AV test challenge with scenic and verifai,” in *2021 IEEE International Conference on Artificial Intelligence Testing, AITest 2021*, pp. 136–142, 2021.
- [8] M. Klischat and M. Althoff, “Generating critical test scenarios for automated vehicles with evolutionary algorithms,” in *2019 IEEE Intelligent Vehicles Symposium, IV 2019*, pp. 2352–2358, 2019.
- [9] Y. Tang, Y. Zhou, Y. Liu, J. Sun, and G. Wang, “Collision avoidance testing for autonomous driving systems on complete maps,” in *IEEE Intelligent Vehicles Symposium, IV 2021*, pp. 179–185, 2021.
- [10] Z. Hu, S. Guo, Z. Zhong, and K. Li, “Coverage-based scene fuzzing for virtual autonomous driving testing,” *CoRR*, vol. abs/2106.00873, 2021.
- [11] D. Kaufmann, L. Klampfl, F. Klück, M. Zimmermann, and J. Tao, “Critical and challenging scenario generation based on automatic action behavior sequence optimization,” in *2021 IEEE International Conference on Artificial Intelligence Testing, AITest 2021*, pp. 118–127, 2021.
- [12] Japan Automobile Manufacturers Association, “Automated driving safety evaluation framework ver 3.0,” tech. rep., December 2022.
- [13] ISO - International Organization for Standardization, “ISO34502: Road vehicles - Test scenarios for automated driving systems - Scenario based safety evaluation framework,” standard, 2022.
- [14] S. Kato et al., “Autoware on board: enabling autonomous vehicles with embedded systems,” in *Proceedings of the 9th ACM/IEEE International Conference on Cyber-Physical Systems, ICCPS 2018*, pp. 287–296, 2018.
- [15] Autoware Foundation, “AWSIM-Labs,” <https://github.com/autowarefoundation/AWSIM-Labs>. Accessed: 2024-09-25.
- [16] Baidu Apollo team (2017), “Apollo: Open Source Autonomous Driving,” <https://github.com/ApolloAuto/apollo>. Accessed: 2024-09-25.
- [17] Y. Tang, Y. Zhou, F. Wu, Y. Liu, J. Sun, W. Huang, and G. Wang, “Route coverage testing for autonomous vehicles via map modeling,” in *IEEE International Conference on Robotics and Automation, ICRA 2021*, pp. 11450–11456, 2021.
- [18] Y. Tang, Y. Zhou, T. Zhang, F. Wu, Y. Liu, and G. Wang, “Systematic testing of autonomous driving systems using map topology-based scenario classification,” in *36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021*, pp. 1342–1346, 2021.
- [19] M. Clavel, F. Durán, S. Eker, P. Lincoln, N. Martí-Ollet, J. Mesequer, and C. L. Talcott, eds., *All About Maude*, vol. 4350 of *Lecture Notes in Computer Science*. Springer, 2007.
- [20] S. Macenski, T. Foote, B. P. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Sci. Robotics*, vol. 7, no. 66, 2022.
- [21] T. Parr, *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2nd ed., 2013.
- [22] N. Wassim, G. J. D., and Y. Mikio, “Pre-crash scenario typology for crash avoidance research,” tech. rep., John A. Volpe National Transportation Systems Center (U.S.), April 2007.
- [23] M. Hiroshi and M. Katsuya, “Various drivers’ braking behaviors and braking distances,” *JES Ergonomics*, vol. Vol.37, No.5, P219–227, no. 5, pp. 219–227, 2001.
- [24] N. Kalra and S. M. Paddock, “Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability?,” *Transportation Research Part A: Policy and Practice*, vol. 94, pp. 182–193, 2016.
- [25] T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, H. Ravanbakhsh, M. Vazquez-Chanlatte, and S. A. Seshia, “VerifAI: A toolkit for the formal design and analysis of artificial intelligence-based systems,” in *Computer Aided Verification, CAV 2019*, vol. 11561, pp. 432–442, 2019.
- [26] D. J. Fremont, E. Kim, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, and S. A. Seshia, “Scenic: a language for scenario specification and data generation,” *Mach. Learn.*, vol. 112, no. 10, pp. 3805–3849, 2023.
- [27] S. Fratini et al., “Verification and validation of autonomous systems with embedded AI: The VIVAS Approach,” in *17th Symposium on Advanced Space Technologies in Robotics and Automation (ASTRA), 2023*, 2023.
- [28] J. Horel et al., “Verifying collision risk estimation using autonomous driving scenarios derived from a formal model,” *J. Intell. Robotic Syst.*, vol. 107, no. 4, p. 59, 2023.
- [29] L. Rummelhard, A. Nègre, and C. Laugier, “Conditional monte carlo dense occupancy tracker,” in *IEEE 18th International Conference on Intelligent Transportation Systems, ITSC 2015*, pp. 2485–2490, 2015.
- [30] A. Gambi, T. Huynh, and G. Fraser, “Generating effective test cases for self-driving cars from police reports,” in *ESEC/SIGSOFT FSE 2019*, pp. 257–267, 2019.
- [31] P. Maul, M. Mueller, and F. Enkler, “Beamng.tech technical paper,” tech. rep., 2021.
- [32] S. K. Bashetty, H. B. Amor, and G. Fainekos, “Deepcrashtest: Turning dashcam videos into virtual crash tests for automated driving systems,” in *2020 IEEE International Conference on Robotics and Automation, ICRA 2020*, pp. 11353–11360, 2020.
- [33] Cyberbotics, “Webots: open-source robot simulator.” <https://github.com/cyberbotics/webots>. Accessed: 2024-09-25.
- [34] A. Gambi, V. Nguyen, J. Ahmed, and G. Fraser, “Generating critical driving scenarios from accident sketches,” in *IEEE International Conference On Artificial Intelligence Testing, AITest 2022, Newark, CA, USA, August 15-18, 2022*, pp. 95–102, 2022.
- [35] H. Shao, L. Wang, R. Chen, H. Li, and Y. Liu, “Safety-enhanced autonomous driving using interpretable sensor fusion transformer,” in *Conference on Robot Learning, CoRL 2022*, pp. 726–737, 2022.
- [36] H. Winner, K. Lemmer, T. Form, and J. Mazzega, *PEGASUS—First Steps for the Safe Introduction of Automated Driving*, pp. 185–195. 01 2019.
- [37] D. Chen and P. Krähenbühl, “Learning from all vehicles,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022*, pp. 17201–17210, 2022.
- [38] J. P. Paardekooper et al., “Automatic identification of critical scenarios in a public dataset of 6000 km of public-road driving,” in *26th International Technical Conference on the Enhanced Safety of Vehicles (ESV)*, no. 19-0255, 2019.
- [39] Association for Standardization of Automation and Measuring Systems (ASAM), “OpenSCENARIO.” <https://www.asam.net/standards/detail/openscenario/v200/>. Accessed: 2024-09-25.
- [40] CARLA team, “ScenarioRunner for CARLA.” https://github.com/carla-simulator/scenario_runner. Accessed: 2024-09-25.
- [41] B. Schütt, T. Braun, S. Otten, and E. Sax, “Sceml: a graphical modeling framework for scenario-based testing of autonomous vehicles,” in *MoDELS ’20: ACM/IEEE 23rd International Conference on Model Driven Engineering Languages and Systems*, pp. 114–120, 2020.
- [42] Association for Standardization of Automation and Measuring Systems (ASAM), “M-SDL: Measurable Scenario Description Language.” https://releases.asam.net/OpenSCENARIO/2.0-concepts/M-SDL_LRM_OS.pdf. Accessed: 2024-09-25.